

# A few models of an uncontrolled inverted pendulum on a cart

J.C. Vaught

EMCH 792: Learning-Based Controls

---

**OBJECTIVE.** The objective of this exercise is to demonstrate that the student (myself) can (1) determine the set of equations needed to model a real, physical system, (2) implement these equations in a Simulink simulation environment in combination with MATLAB, and (3) implement the same system in Simscape’s Multibody simulation environment. Although unspecified, it seems prudent to assume that the reader is also looking for a comparison of the accuracies of both simulation methodologies — particularly seeing as the rubric requires a methodology, results, and discussion section.

---

## METHODOLOGY.

Here, the pendulum is assumed to be a massless rod, with a length of  $\ell$  and a point mass  $m$  at the end. The end of the pendulum opposite the point mass is attached to the cart, which has a mass of  $M$  and rides on a frictionless horizontal track. The attachment to the cart is assumed to have some amount of friction, which is modelled as a viscous rotational damper with a coefficient of  $c_\theta$ . The positive directions follow the arrows included in the assignment handout.

The equations of motion are derived using Lagrange’s equations, with the cart position  $x$  and the pendulum angle  $\theta$  as the generalized coordinates. Skipping the intermediate algebra, the resulting equations of motion are given by

$$(M + m)\ddot{x} + m\ell \cos(\theta)\ddot{\theta} - m\ell \sin(\theta)\dot{\theta}^2 = 0,$$

and

$$m\ell \cos(\theta)\ddot{x} + m\ell^2\ddot{\theta} + c_\theta\dot{\theta} - mgl \sin(\theta) = 0.$$

## SIMULATION STEPS.

The Simulink model is built by entering the equations of motion into a MATLAB function block. The reader can refer to the appendix if they are so inclined to review the code for this block. During run-time, the solver uses the current values of  $\theta$  and  $\dot{\theta}$  to solve the coupled  $2 \times 2$  linear system for the cart acceleration and the angular acceleration. Then, a ‘two-stage’ or dually-cascaded integrator is used for both  $\theta$  and cart position to get velocity and position for both, closing the feedback loop. An output block stores the data for plotting.

The sister Simscape Multibody model is made in a much simpler fashion — just click, drag, and drop

the physical components, with no equation derivation required. The model is built in the following manner: add a world frame for the universal coordinate system, add a mechanism configuration block to create gravity, add a prismatic joint to make the cart move horizontally, add a solid body to be the cart mass, add a revolute joint with viscous rotational damping to attach the pendulum, add a rigid transform for the pendulum length, and add a point mass. Finally, motion sensors are added to measure the cart position and pendulum angle for export. The ‘weird’ world ruler block is not as scary as it sounds. It is simply a block that contains a system inside it to draw a ruler in the mechanics explorer as a visual aid.

Both models are initialized with all values set to 0, except for  $\theta(0) = 5^\circ$  since we need a small initial push. A step size of 0.01 seconds is used for both models. The models are run for both 10 and 200 seconds to show the short term response and the long term damping response. The logged time histories are exported from MATLAB to a JSON file, since I prefer using Typst and Lilaq to handle plotting, which imports the data directly.

## RESULTS.

Figures 1 and 2 show the first 10 seconds of the response, and Figures 3 and 4 extend the same signals to 200 seconds. In every figure, the Simscape trace (solid) and Simulink trace (dashed) lie directly on top of one another.

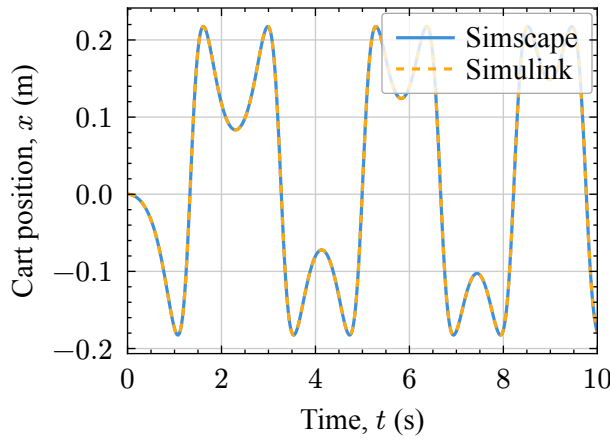


Figure 1: Cart position over 10 s.

The cart oscillates back and forth with an amplitude of roughly 0.2 m, driven entirely by the reaction force at the pivot as the pendulum swings.

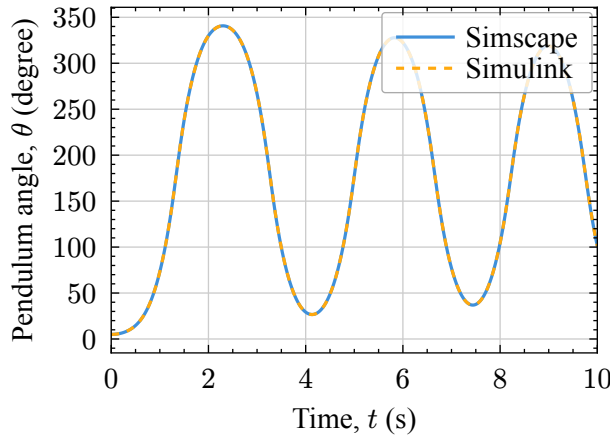


Figure 2: Pendulum angle over 10 s.

The pendulum falls away from the unstable upright position, over-swings well past the hanging position, and then begins oscillating about  $\theta = 180^\circ$ .

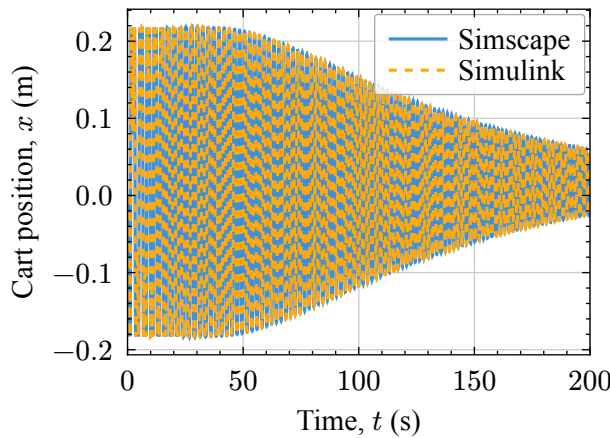


Figure 3: Cart position over 200 s.

Over the long run, the cart never drifts away. It oscillates about a fixed mean while the amplitude slowly shrinks.

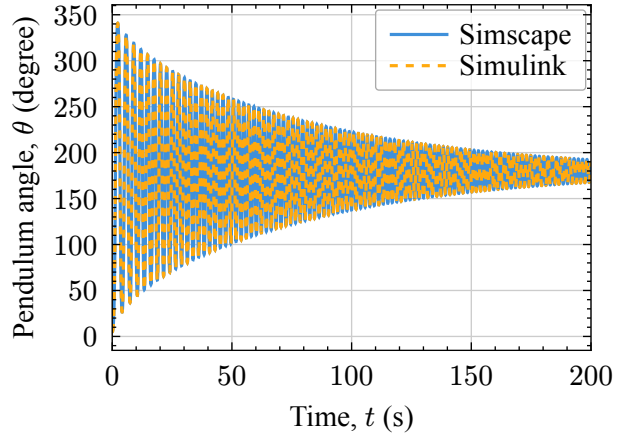


Figure 4: Pendulum angle over 200 s.

## DISCUSSION.

The short-term behavior is exactly what one would expect from an uncontrolled inverted pendulum. The upright position is unstable, so the small  $5^\circ$  perturbation grows and the pendulum falls, as seen in Figures 1 and 2.

It can also be seen that the two models are nearly identical, if not exactly so. The hand-derived equations of motion and the click-and-drag physical model describe the same dynamics, so an error in either one would have shown up as a visible gap between the solid and dashed traces.

Additionally, it is clearly seen that as the pendulum swings to and fro, the cart gets pulled along with it. This is a direct consequence of conservation of momentum. No external horizontal force acts on the system, so the center of mass stays fixed and the cart oscillates about a constant mean. Meanwhile, the viscous damper at the pivot slowly bleeds energy out of the swing, so the pendulum settles toward the hanging position at  $\theta = 180^\circ$  in Figure 4. The decay takes the full 200 seconds because  $c_\theta$  is small.

## APPENDIX.

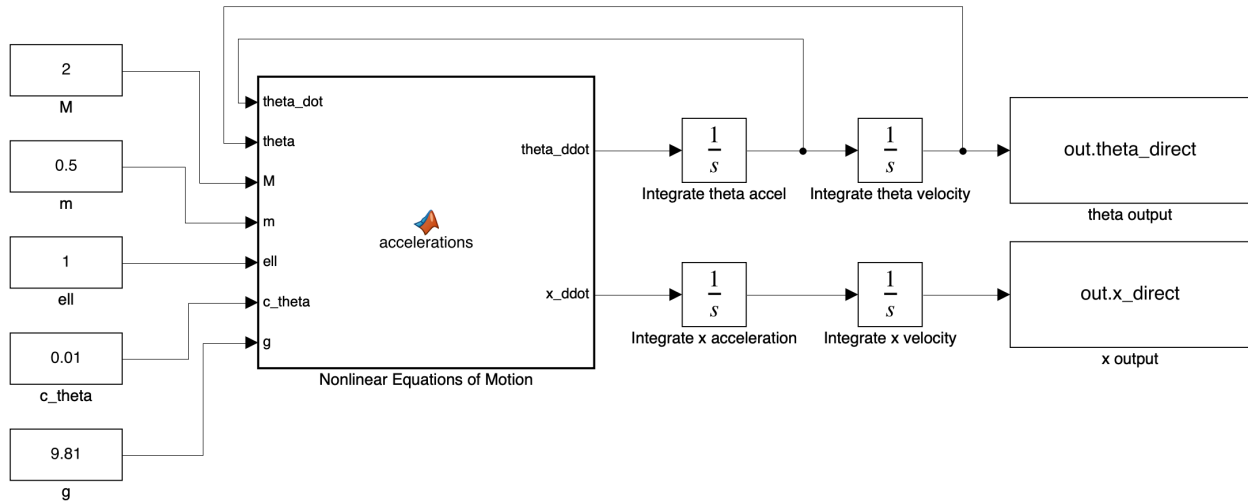


Figure 5: Editable Simulink implementation of the nonlinear mass-matrix equations.

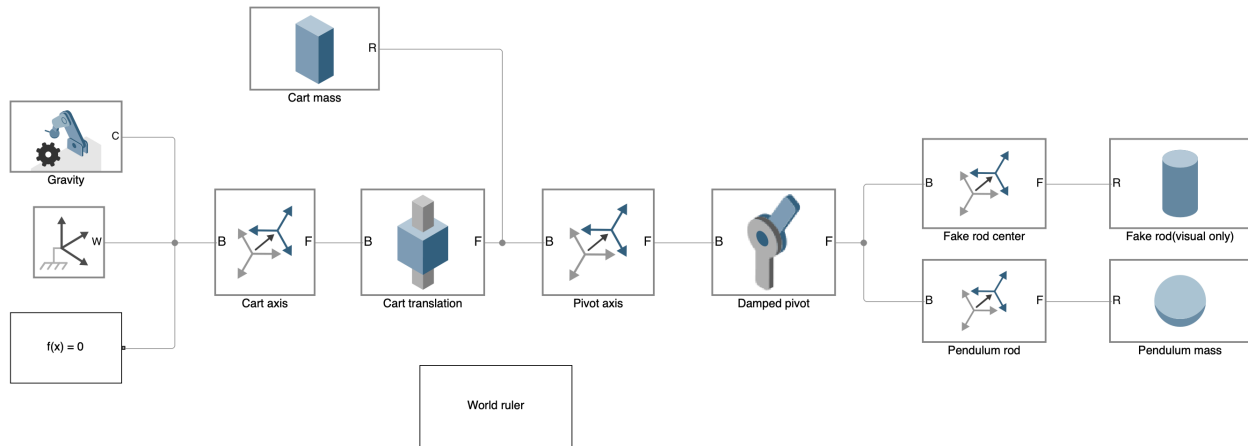


Figure 6: Simscape Multibody block diagram.

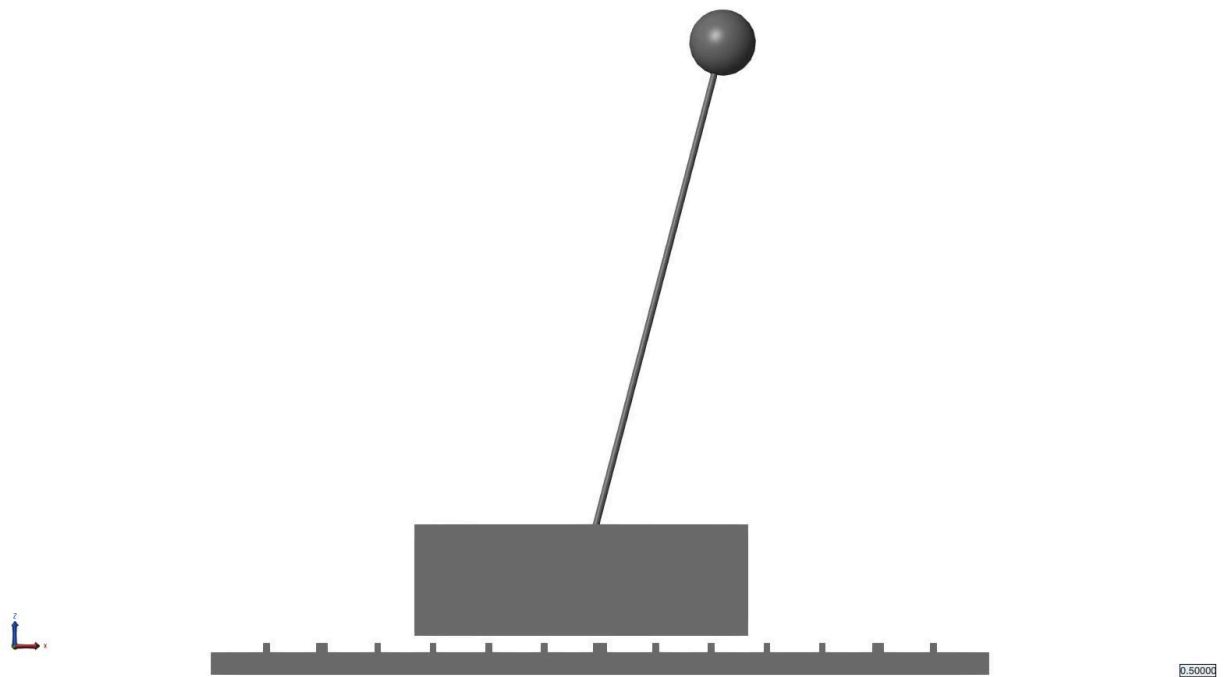


Figure 7: Mechanics Explorer view of the cart, ruler, pendulum rod, and point mass.

This is the custom MATLAB function block used in Simulink model.

```
function [theta_ddot, x_ddot] = accelerations(theta_dot, theta, M, m, ell, c_theta, g)
% Solve the coupled nonlinear equations for the two accelerations.

s = sin(theta);
c = cos(theta);

mass_matrix = [M + m, m * ell * c; m * ell * c, m * ell^2];
forcing = [m * ell * s * theta_dot^2; m * g * ell * s - c_theta * theta_dot];

acceleration = mass_matrix \ forcing;
x_ddot = acceleration(1);
theta_ddot = acceleration(2);
end
```